A Complete Guide to AWS ECS (Elastic Container Service)

medium.com/@amogh.r.hegdeln21/a-complete-guide-to-aws-ecs-elastic-container-service-24bc3a4c8728

Amogh R Hegdeln May 2, 2026

If you're working with containers on AWS and want something simpler than Kubernetes, **Amazon ECS (Elastic Container Service)** is one of the best options available. It removes the need to manage a control plane while still giving you flexibility to run containers at scale.

In this blog, we'll go from fundamentals to advanced concepts including **internals like network modes and namespaces,** so you not only deploy ECS services but also understand what's happening under the hood.

🧠 What is Amazon ECS?

Amazon ECS (Elastic Container Service) is a fully managed container orchestration service provided by AWS that allows you to **deploy, manage, scale, and run containerized applications** without having to operate your own orchestration layer.

At a high level, ECS takes care of scheduling containers on compute infrastructure, maintaining application availability, handling scaling, and integrating tightly with other AWS services like networking, IAM, logging, and load balancing.

It is commonly used to run **microservices, APIs, background workers, batch jobs, and long-running applications** in a highly available and scalable way.

Ways to run containers in ECS

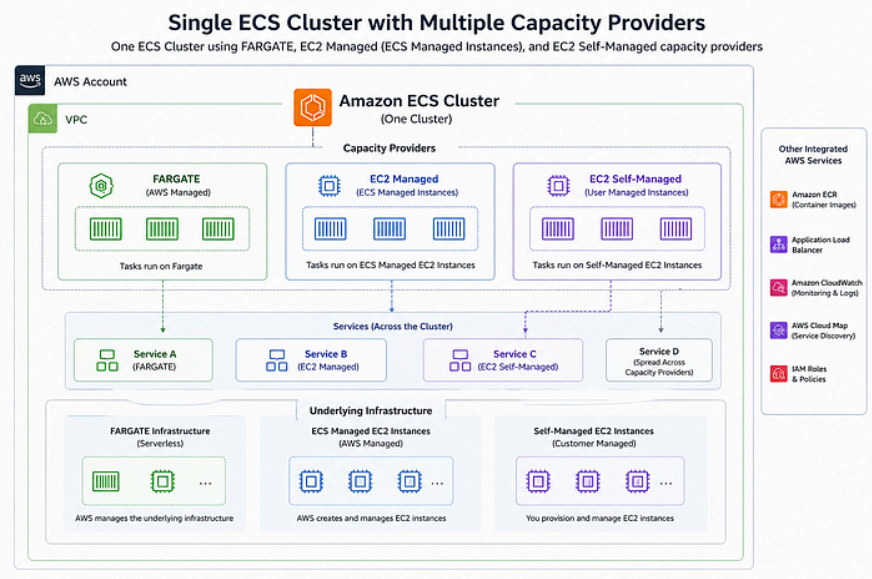
ECS gives you flexibility in how you run workloads depending on how much control you want over infrastructure.

Fargate (Serverless) — you don't think about creating or managing servers. Great for most common workloads

Managed instances — Amazon ECS will manage patching and scaling on your behalf while giving you configurability about the types of instances. Great for more advanced workloads.

Self-managed instances — you must ensure the instances are patched and scaled properly, and you have full control over the instances.

Press enter or click to view image in full size

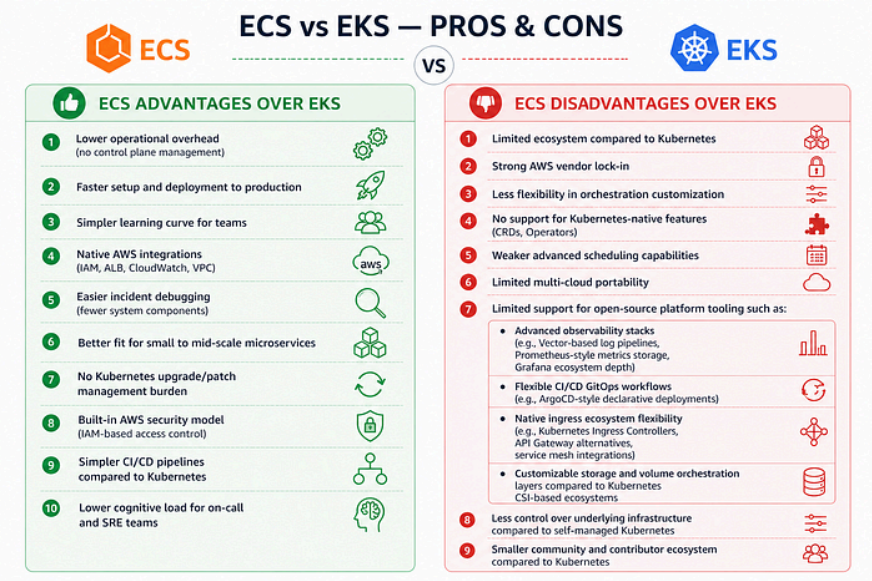


ECS vs EKS

Every tool and technology comes with its own strengths and limitations. As architects, the goal is not to label any solution as inherently good or bad, but to evaluate how well it aligns with a specific use case.

A technology is only as effective as its fit for the problem it is solving. Therefore, developing a deep, end-to-end understanding of how each tool works enables us to make informed architectural decisions and choose the most appropriate solution for the given requirements.

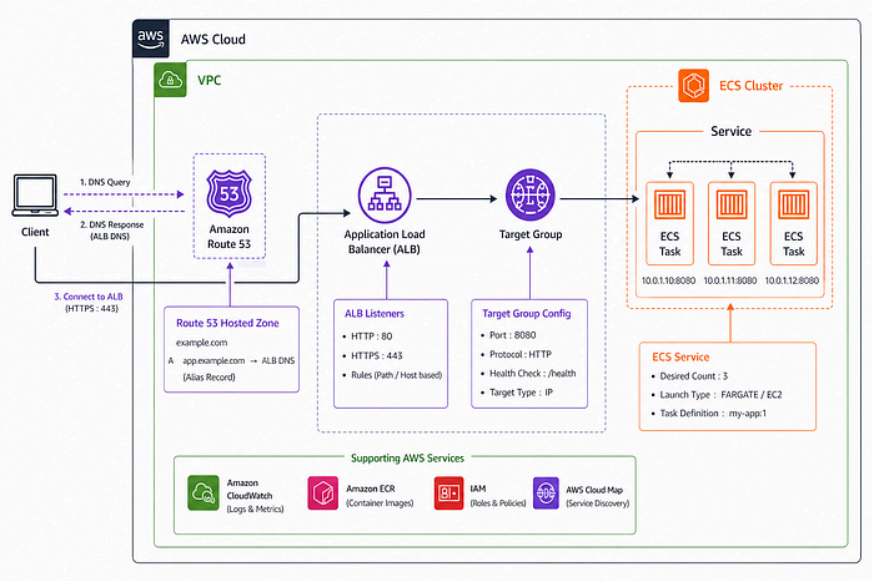
Press enter or click to view image in full size



Now let's deploy a simple web application on ECS 🚀

High-level architecture overview

Press enter or click to view image in full size



Step 1: Set Up an ECS Cluster

Begin by creating an **Amazon Elastic Container Service (ECS) cluster**. This cluster will serve as the environment for deploying and managing your containerized applications.

```
aws ecs create-cluster \
    --cluster-name MyFargateCluster \
    --capacity-providers FARGATE \
    --settings name=containerInsights,value=enabled
```

> Behind the scenes, a CloudFormation template will be executed to set up this ECS cluster.

Step 2: Create a Private ECR Repository for Your Container Images

Next, set up a **private Amazon Elastic Container Registry (ECR)** repository. This will be the secure location where you'll push and store your container images before deploying them to ECS.

```
aws ecr create-repository \
    --repository-name hello-service \
    --encryption-configuration encryptionType=AES256
```

Step 3: Create the ECS Task Execution Role

Create an **ECS Task Execution Role**. This IAM role grants ECS the necessary permissions to pull container images from ECR, log to CloudWatch, and interact with other AWS services required by your ECS tasks.

Create Trust Policy file `ecs-task-trust-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ecs-tasks.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Create ecsTaskExecutionRole Role:

```
aws iam create-role \
  --role-name ecsTaskExecutionRole \
  --assume-role-policy-document file://ecs-task-trust-policy.json

# Attach Policy

aws iam attach-role-policy \
  --role-name ecsTaskExecutionRole \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AmazonECSTaskExecutionRolePolicy
```

Step 4: Build and Containerize Your Web Application

Develop a simple **web application**, then **containerize** it using Docker or another containerization platform. Once your app is containerized, **push the image** to the ECR repository you created in Step 2. This step prepares your application for deployment on ECS.

```
# vim app.py

from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "hello"

app.run(host="0.0.0.0", port=3000)
```

```
# vim requirements.txt

flask
```

```
# vim Dockerfile

FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 3000

CMD ["python", "app.py"]
```

```
podman build -t hello-service .
```

```
export AWS_REGION=ap-south-1
export AWS_ACCOUNT_ID=123456789012
export REPO_NAME=hello-service
```

```
# Login to ECR registry

aws ecr get-login-password --region $AWS_REGION | \
podman login --username AWS --password-stdin \
$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
```

```
# Tag container image

podman tag hello-service:latest \
$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/hello-
service:latest
```

```
# Push image to ECR repo

podman push \
$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/hello-
service:latest
```

Step 5: Deploy the Application as an ECS Service

After pushing your container image to ECR, deploy the application by creating an **ECS service**. This service will run and manage your containerized application on the ECS cluster, ensuring that the desired number of tasks (containers) are running continuously, and it will automatically handle scaling and load balancing if needed.

Create file `task-def.json`

```
{
  "family": "hello-service-task",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "arn:aws:iam::
<ACCOUNT_ID>:role/ecsTaskExecutionRole",
  "containerDefinitions": [
    {
      "name": "hello-service",
      "image": "<ACCOUNT_ID>.dkr.ecr.
<REGION>.amazonaws.com/hello-service:latest",
      "essential": true,
      "portMappings": [
        {
          "containerPort": 3000,
          "protocol": "tcp"
        }
      ]
    }
  ]
}
```

Register Task Definition

```
aws ecs register-task-definition \
  --cli-input-json file://task-def.json
```

Create an ALB and Task Security Group

```
export VPC_ID=<your-vpc-id>

# Create ALB Security Group

aws ec2 create-security-group \
  --group-name hello-alb-sg \
  --description "ALB Security Group" \
  --vpc-id $VPC_ID

# Copy the output GroupId

export ALB_SG_ID=<alb-sg-id>

# Allow inbound HTTP (port 80 from internet)

aws ec2 authorize-security-group-ingress \
  --group-id $ALB_SG_ID \
  --protocol tcp \
  --port 80 \
  --cidr 0.0.0.0/0

# Create ECS Task Security Group

aws ec2 create-security-group \
  --group-name hello-ecs-sg \
  --description "ECS Task Security Group" \
  --vpc-id $VPC_ID

# Copy the output GroupId

export ECS_SG_ID=<ecs-sg-id>

# Allow ALB → ECS communication

aws ec2 authorize-security-group-ingress \
  --group-id $ECS_SG_ID \
  --protocol tcp \
  --port 3000 \
  --source-group $ALB_SG_ID
```

Create Target Group (ALB → ECS port 3000)

```
aws elbv2 create-target-group \
  --name hello-service-tg \
  --protocol HTTP \
  --port 3000 \
  --vpc-id <VPC_ID> \
  --target-type ip \
  --health-check-path /
```

Create ALB

```
aws elbv2 create-load-balancer \
  --name hello-service-alb \
  --subnets <SUBNET_1> <SUBNET_2> \
  --security-groups $ALB_SG_ID \
  --scheme internet-facing \
  --type application
```

Update **SUBNET_1** and **SUBNET_2** with the actual public subnet IDs. Ensure that the public subnets are in the same availability zones as the private subnets, so the ECS tasks remain accessible.

Create Listener (ALB :80 → Target Group)

```
aws elbv2 create-listener \
  --load-balancer-arn <ALB_ARN> \
  --protocol HTTP \
  --port 80 \
  --default-actions Type=forward,TargetGroupArn=<TG_ARN>
```

Path-based routing rule (`/ → service`)

(Already default route, but explicit rule if needed)

```
aws elbv2 create-rule \
  --listener-arn <LISTENER_ARN> \
  --priority 1 \
  --conditions Field=path-pattern,Values="/" \
  --actions Type=forward,TargetGroupArn=<TG_ARN>
```

ECS Service (connect everything)

```
aws ecs create-service \
  --cluster <CLUSTER_NAME> \
  --service-name hello-service \
  --task-definition hello-service-task \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=
[<SUBNET_1>,<SUBNET_2>],securityGroups=
[$ECS_SG_ID],assignPublicIp=DISABLED}" \
  --load-balancers "targetGroupArn=<TG_ARN>,containerName=hello-
service,containerPort=3000"
```

Update **SUBNET_1** and **SUBNET_2** with the actual private subnet IDs. Ensure that these private subnets are in the same availability zones as the public subnets used during the ALB creation step.

Deploying a Background Service and Enabling Service-to-Service Communication

Let's say you want to deploy an additional ECS service called **background-service** within the same ECS cluster, and you want it to communicate with the **hello-service** via DNS (e.g., `hello-service.example.com`). To achieve this, we need to set up **service discovery** in ECS, which allows services to resolve each other by DNS name.

Setting Up Service Discovery:

Create a Namespace and Associate Namespace with VPC

```
aws servicediscovery create-private-dns-namespace \
  --name example.com \
  --vpc <VPC_ID> \
  --region ap-south-1
```

Replace `<VPC_ID>` with the VPC ID corresponding to the subnets selected during ALB creation and ECS service deployment.

```
#Get the namespace id and store it in env varibale

aws servicediscovery list-namespaces

# Output

{
  "Name": "example.com",
  "Id": "ns-xxxxx"
}

# Store this id in ENV variable

export NAMESPACE_ID=ns-xxxxx
```

Create Service Discovery Service

```
aws servicediscovery create-service \
  --name hello-service \
  --dns-config "NamespaceId=$NAMESPACE_ID,DnsRecords=
[{Type=A,TTL=60}]" \
  --health-check-custom-config FailureThreshold=1

# Get the service discover Arn value from the output and store it
in env varibale

aws servicediscovery list-services

export SERVICE_DISCOVERY_ARN=<Arn_value>
```

Update the existing **hello-service** to enable service discovery so that it can be accessed via the DNS name **hello-service.example.com** by other tasks running within the same cluster.

```
aws ecs update-service \
  --cluster <CLUSTER_NAME> \
  --service hello-service \
  --service-registries "registryArn=$SERVICE_DISCOVERY_ARN"
```

When deploying the **background-service**, ensure that you pass the **DNS name** (`hello-service.example.com`) as an environment variable in the **background-service task definition**. This will allow the background service to reference and resolve the DNS name of **hello-service** for service-to-service communication.

🔍 Behind the Scenes: How ECS Service Discovery Resolves IPs

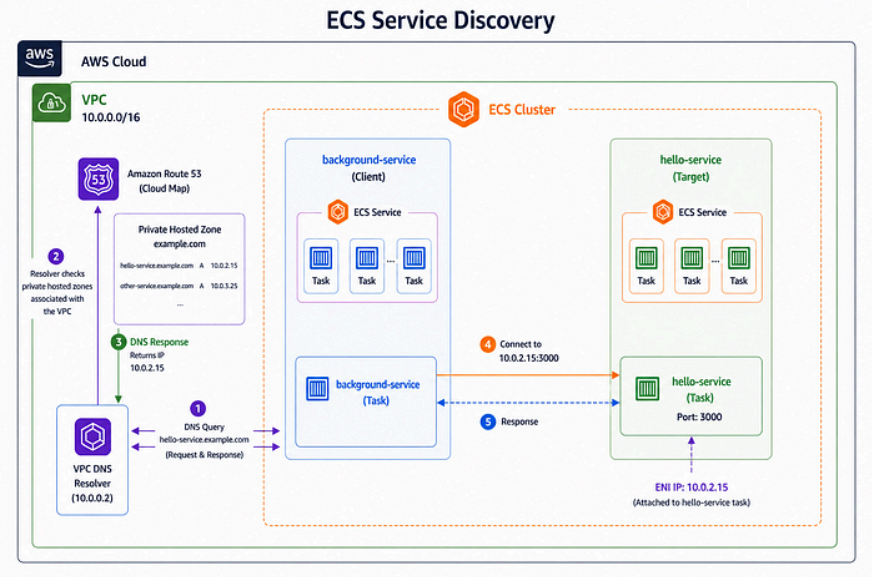
Behind the scenes, ECS service discovery relies on the VPC DNS resolver. Each ECS task uses the resolver IP defined in `/etc/resolv.conf`, which typically points to the VPC DNS resolver address (the base VPC CIDR range (`10.0.0.0/16`) plus two, for example `10.0.0.2`)

When a service name like `hello-service.example.com` is queried, the request is sent to this VPC resolver. The resolver then checks all **private hosted zones** associated with the VPC. Since AWS Cloud Map creates a private hosted zone for the namespace and associates it with your VPC, the resolver looks for the corresponding DNS record under the `example.com` hosted zone.

If a matching record is found (for example, `hello-service.example.com`), the resolver returns the private IP address of the ECS task that registered itself under that name. This IP corresponds to the ENI attached to the task.

This mechanism enables seamless service-to-service communication within the VPC using simple DNS names, without requiring an internal load balancer.

Press enter or click to view image in full size



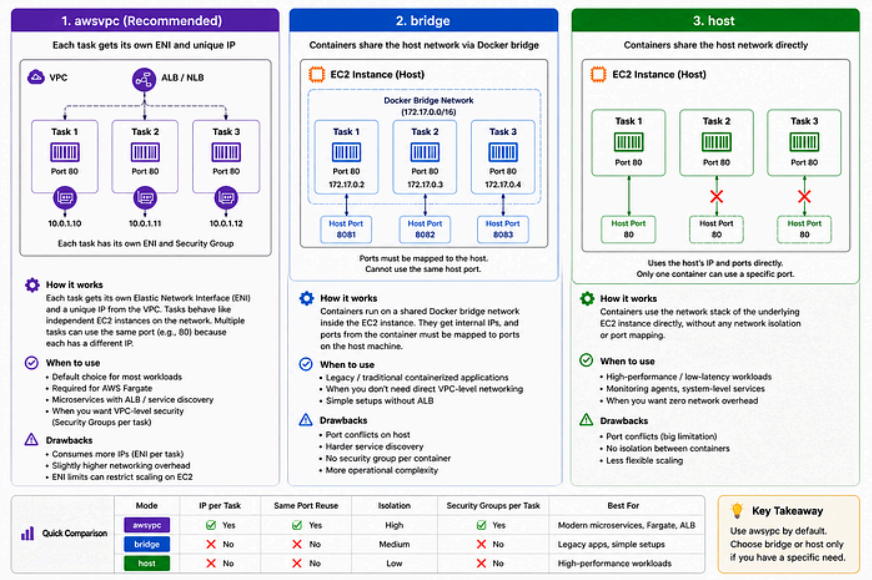
AWS ECS Network Modes

Amazon ECS supports multiple networking modes that define how containers communicate within a cluster and with external services.Choosing the right mode impacts scalability, security, and how ports and IP addresses are

managed.

The three primary modes **awsvpc, bridge, and host** offer different trade-offs between isolation, simplicity, and performance. Understanding these differences helps you design more efficient and reliable containerized applications.

Press enter or click to view image in full size



⚖️ ECS Compute Options: Fargate vs EC2 (Managed vs Self-Managed Capacity)

When deploying applications on **Amazon ECS**, one of the most important decisions is how you want to run your container workloads, using **AWS Fargate (fully managed compute)** or **EC2-backed capacity (managed or self-managed via Auto Scaling Groups)**.

With Fargate, AWS manages provisioning, scaling, and infrastructure maintenance, allowing you to focus entirely on your application, but it comes with limited control over the underlying resources. In contrast, EC2-based deployments give you deeper control over instance types, networking, and resource utilization, but require you to manage scaling, patching, and capacity planning.

Understanding this trade-off is key to designing an ECS architecture that balances **simplicity, cost efficiency, and flexibility** based on your workload needs.

When deploying applications on Amazon ECS using EC2, one of the critical decisions you'll face is choosing how to scale your infrastructure: should you use **Managed Instances** or **Self-Managed Auto Scaling Groups (ASG)**? This decision has significant implications for cost, control, and operational complexity.

Managed vs. Self-Managed Scaling: The Fundamental Difference

Infrastructure-Aware Scaling (Managed Instances):

In the **Managed Instances** model, Amazon ECS takes care of both the scaling of instances and the provisioning process. This tightly coupled setup means ECS understands exactly what your tasks need and directly provisions the appropriate infrastructure.

- The , based on your task definition.
- When ECS schedules a task, it checks the task's requirements (such as `cpuArchitecture`, like `ARM64` or `X86_64`) and makes direct API calls to provision the right type of EC2 instance.
- ECS is responsible for "Just-in-Time" provisioning, meaning it can launch exactly the right type of instance (e.g., an ARM64 instance) without you worrying about the underlying infrastructure.

Capacity-Aware Scaling (Self-Managed ASG):

In the **Self-Managed ASG** approach, the **EC2 Auto Scaling Group (ASG)** handles the instance scaling independently of ECS. While ECS triggers scaling events, it does not have direct control over the types of EC2 instances launched.

Who Makes the Decision? The **EC2 Auto Scaling Group**.

How it Works:

- ECS schedules a task (e.g., an `ARM64` task).
- ECS updates a metric, `CapacityProviderReservation`, to signal the need for more resources.
- The looks at this metric, sees the increase, and scales up.

The Blind Spot: The ASG doesn't know about the ECS task requirements. It only refers to its **Launch Template** for instance types, and may end up launching an **x86 instance** when ECS needs an **ARM64 instance**.

Potential Issues: If your ASG contains a mix of different instance types or architectures (e.g., `x86_64` and `ARM64`), ECS tasks may end up in the **pending** state while waiting for the correct instance type to be launched. This delay happens because the ASG will launch an instance that doesn't meet the task's requirements, causing ECS to wait until the correct one is provisioned. Since it's a self-managed ASG, you would need to update the AMI in the Launch Template and refresh the ASG during ECS upgrades. If you use a **single mixed-instance ASG**, you might run into same issues where, for example, a GPU-based instance gets terminated, but a CPU-based instance is launched instead, causing tasks to wait due to mismatched resource requirements.

How to Make Self-Managed ASGs "Smart"

To avoid misalignments and improve deployment reliability, you can optimize your **Self-Managed ASG** setup by **segregating** your instances based on task requirements.

1. Create Multiple ASGs for Different Architectures and Instance Types

If you're using multiple instance types or architectures (e.g., `ARM64` vs `x86`), separate them into different ASGs. This way, each ASG is **dedicated to a specific architecture or instance type**:

- For ARM64 instances (e.g., `t4g.medium`, `m6g.large`).
- For x86 instances (e.g., `t3.medium`, `m5.large`).
- For Spot instances to reduce cost (e.g., `t3a.medium`, `c5.large` Spot instances).

2. Use Capacity Provider Strategy for Placement Control

With **Capacity Provider Strategy**, you can define how ECS should distribute tasks across your ASGs, making sure that tasks are launched in the correct ASG based on their requirements. For instance, you can define weighted placements, so ECS knows where to place tasks based on architecture, instance type, or even cost optimization (Spot vs On-demand).

Example:

- Deploy a service with 10 replicas, evenly distributing the load with 50% weight on Spot instances and 50% on On-demand instances. This setup ensures no complete downtime in case of a Spot interruption, as 5 tasks will

continue running on On-demand instances, while still saving costs with 5 tasks running on Spot instances.

This allows you to achieve cost savings while ensuring that your service is always up and running.

Key Takeaway:

- are great for simplicity and automation, but they limit your ability to control cost and instance selection.
- gives you full control over instance types and pricing, but requires careful configuration to ensure that ECS tasks are placed on the correct instance type.